In [21]:
```
import matplotlib.pyplot as plt
import cv2
import torchinfo
from torch import nn
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
import PIL.Image as Image
from pathlib import Path
import time
```

In [22]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
```

```
cuda
```

Dataset downloading, preparation and visualization

Dataset should be around 100 images of natural photography that we need to gray scale and resize to 320x240 pixels.
The images are from CoCo train dataset and are in the folder dataset.

Homography explains how a flat image is transformed in other flat image if you look the image from different angle.

In [ ]:

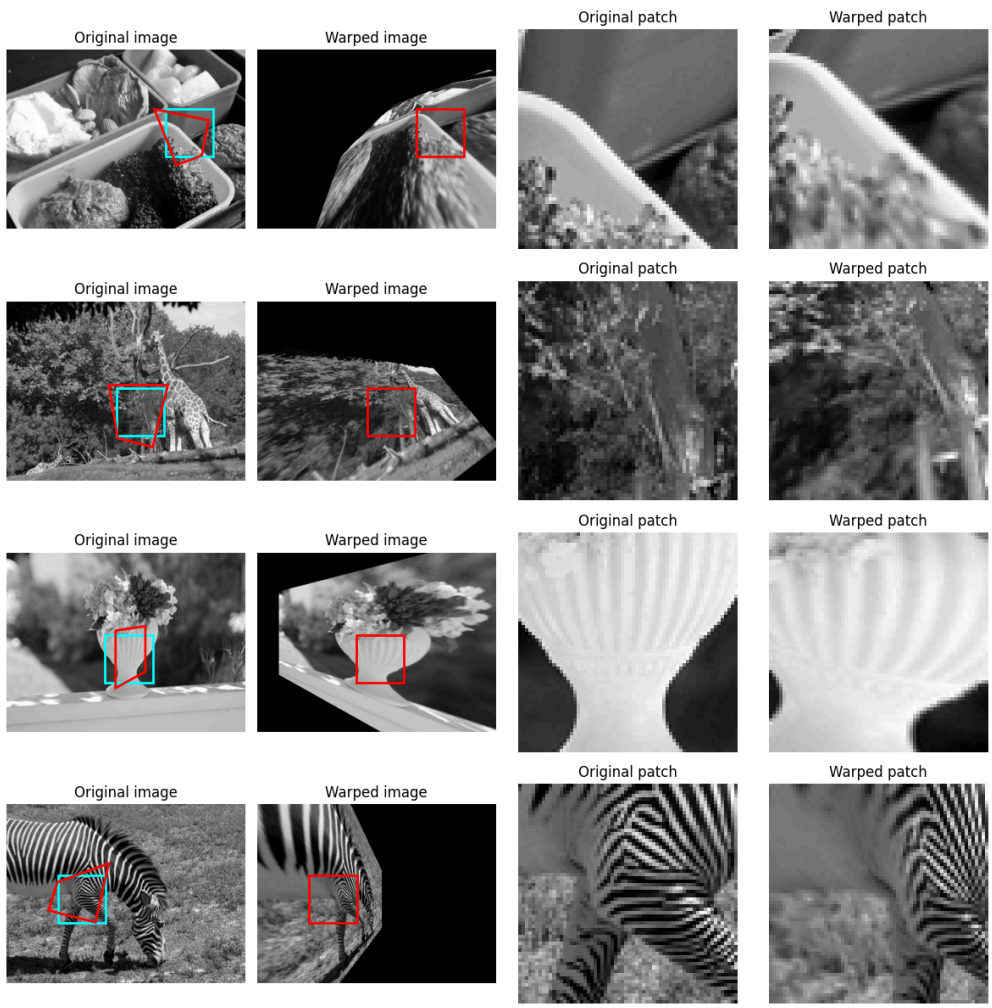
```
class HomographyPreloadDataset(Dataset):
    def __init__(self, root = './dataset', resize=(320,240), patch_size=64,
max_shift=16):
        self.images_paths = list(Path(root).glob('*.*'))
        self.resize = resize
        self.patch_size = patch_size
        self.max_shift = max_shift
        self.images = []

        print(f'Preloading {len(self.images_paths)} samples into RAM...')

        for i, image_path in enumerate(self.images_paths):
            image = self.load_image(image_path, resize)
            self.images.append(image)
            if (i + 1) % 10 == 0:
                print(f"  {i + 1}/{len(self.images_paths)} loaded...")

        print(f'Done ')


    def load_image(self, image_path, resize)-> np.ndarray:
        image = Image.open(image_path).convert('L').resize(resize)
        return np.array(image, dtype=np.float32) / 255

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        image = self.images[idx]
        data = self.generate_image(image)
        return (
            data['sample'], # (2, 64, 64)
            data['offsets'],
            data['base_corners'],
            data['moved_corners'],
            data['H_mat'],
            data['warped_image'],
        )

    def get_random_top_left_corner(self, image_w, image_h) -> tuple[int,
int]:
        safe_shift = self.max_shift + 20
        x = np.random.randint(safe_shift, image_w - self.patch_size -
safe_shift)
        y = np.random.randint(safe_shift, image_h - self.patch_size -
safe_shift)
        return x, y

    def get_window_corners(self, x, y):
        top_left = [x, y]
        top_right = [x + self.patch_size -1, y]
        bottom_right = [x + self.patch_size - 1, y + self.patch_size -1]
        bottom_left = [x, y + self.patch_size - 1]
```

```
        return np.array([top_left, top_right, bottom_right, bottom_left],
dtype=np.float32)

    def get_random_offsets(self):
        return np.random.randint(-self.max_shift, self.max_shift +1, size=
(4,2)).astype(np.float32)

    def generate_image(self, image):
        image_H, image_W = image.shape

        x, y = self.get_random_top_left_corner(image_W, image_H)
        base = self.get_window_corners(x, y)
        offsets = self.get_random_offsets()
        moved = base + offsets

        H_mat = cv2.getPerspectiveTransform(base, moved)

        H_mat_inv = np.linalg.inv(H_mat) # invert H to get H_inv

        warped_image = cv2.warpPerspective(image, H_mat_inv, (image_W,
image_H))
        patch = image[y:y + self.patch_size, x:x + self.patch_size]
        warped_patch = warped_image[y:y + self.patch_size, x:x +
self.patch_size]
        two_channel = np.stack([patch, warped_patch], axis=0)

        return {
            "sample":        torch.FloatTensor(two_channel),  # (2, 64, 64)
            "offsets":       torch.FloatTensor(offsets), # (4, 2)
            "base_corners":  torch.FloatTensor(base), # (4, 2)
            "moved_corners": torch.FloatTensor(moved), # (4, 2)
            "H_mat":         torch.FloatTensor(H_mat), # (3, 3)
            "warped_image":  warped_image, # (240, 320)
        }
```

In [24]:
```
def visualize_samples(dataset, n=4):
    fig, axes = plt.subplots(n, 4, figsize=(12, n * 3))

    for i in range(n):
        image = dataset.images[i]
        data = dataset.generate_image(image)

        sample        = data['sample']
        base_corners  = data['base_corners'].numpy()
        moved_corners = data['moved_corners'].numpy()
        x = int(base_corners[0, 0])
        y = int(base_corners[0, 1])

        patch        = sample[0].numpy()
        warped_patch = sample[1].numpy()
        warped_image = data['warped_image']

        # original image with cyan square + red skewed polygon
        axes[i, 0].imshow(image, cmap='gray')
        cyan_rect = plt.Rectangle((x, y), dataset.patch_size,
dataset.patch_size,linewidth=2, edgecolor='cyan', facecolor='none')
        axes[i, 0].add_patch(cyan_rect)
        red_poly = plt.Polygon(moved_corners, linewidth=2, edgecolor='red',
facecolor='none', closed=True)
        axes[i, 0].add_patch(red_poly)
        axes[i, 0].set_title('Original image')
        axes[i, 0].axis('off')

        # warped image with red perfect square
        axes[i, 1].imshow(warped_image, cmap='gray')
        rect = plt.Rectangle((x, y), dataset.patch_size,
dataset.patch_size,
                              linewidth=2, edgecolor='red',
facecolor='none')
        axes[i, 1].add_patch(rect)
        axes[i, 1].set_title('Warped image')
        axes[i, 1].axis('off')

        # original patch (cyan region)
        axes[i, 2].imshow(patch, cmap='gray')
        axes[i, 2].set_title('Original patch')
        axes[i, 2].axis('off')

        # warped patch (red region)
        axes[i, 3].imshow(warped_patch, cmap='gray')
        axes[i, 3].set_title('Warped patch')
        axes[i, 3].axis('off')

    plt.tight_layout()
    plt.show()
```

In [25]:
```
dataset = HomographyPreloadDataset()
dataloader = DataLoader(dataset, batch_size=64, shuffle=True,
num_workers=0, pin_memory=True, )
visualize_samples(dataset, n=4)
```

```
Preloading 100 samples into RAM...
  10/100 loaded...
  20/100 loaded...
  30/100 loaded...
  40/100 loaded...
  50/100 loaded...
  60/100 loaded...
  70/100 loaded...
  80/100 loaded...
  90/100 loaded...
  100/100 loaded...
Done
```



Neural Networks

Building 2 neural networks that will share a backbone that will be built from ResNet blocks.

ResNet block

ResNet block is built from Conv - BatchNorm - ReLu - Conv - BatchNorm - ReLu. Its key idea is the skip connection (residual connection), where the input is added to the output of the block.

If the number of input and output channels is different, a 1×1 convolution is used to match dimensions before the addition.

This skip connection helps prevent vanishing gradients and makes training deep networks more stable. The skip connection preserves the original signal and improves gradient flow, making it easier for the network to learn useful transformations.

In [26]:

```
class ResNetBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.convolution1 = nn.Conv2d(in_channels, out_channels,
kernel_size=3, padding="same", bias=False)
        self.batchNorm1 = nn.BatchNorm2d(out_channels)
        self.convolution2 = nn.Conv2d(out_channels, out_channels,
kernel_size=3, padding="same", bias=False)
        self.batchNorm2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if in_channels != out_channels:
            self.identity = nn.Conv2d(in_channels, out_channels,
kernel_size=1)
        else:
            self.identity = nn.Identity()

    def forward(self, x):
        identity = self.identity(x)
        out = self.convolution1(x)
        out = self.batchNorm1(out)
        out = self.relu(out)
        out = self.convolution2(out)
        out = self.batchNorm2(out)
        out += identity
        out = self.relu(out)

        return out
```

Backbone structure
The sharing backbone has 3x (ResNet block -> MaxPool) and at the end 2x ResNet blocks ant then flatten is called to convert a 3D tensor into a 1D vector so it can be fed into Linea. Linear layer compresses the spatial features into a compact 512 dimensional feature vector that heads can use

After the convolutional layer the network has learned spatial features (specific parts in the image). The linear layer combines them all in a signal global description.
The regression or classification head will take that 512 dim summary and will predict the 8 corner offsets from it

In [27]:
```
class BackboneBody(nn.Module):
    def __init__(self, in_channels):
        super().__init__()

        self.body = nn.Sequential(
            ResNetBlock(in_channels, 64),
            ResNetBlock(64, 64),
            nn.MaxPool2d(kernel_size= 2, stride=2),

            ResNetBlock(64, 64),
            ResNetBlock(64, 64),
            nn.MaxPool2d(kernel_size= 2, stride=2),

            ResNetBlock(64, 128),
            ResNetBlock(128, 128),
            nn.MaxPool2d(kernel_size= 2, stride=2),

            ResNetBlock(128, 128),
            ResNetBlock(128, 128),

            nn.Flatten(), # (128, 8, 8) -> (8192)
            nn.Linear(8192, 512), # most sig vals
            nn.ReLU(inplace=True),
        )

    def forward(self,x):
        return self.body(x)
```

Regression head
The regression head takes the 512 dimensional feature vector from the backbone and directly predicts the 8 corners offsets (4 corners x x,y) as continuoues values. While training uses MSE/RMSE loss comparing predicted offsets to the ground truth offsets that are generated during dataset creation

In [28]:

```
class RegressionHead(nn.Module):
    def __init__(self):
        super().__init__()
        self.body = BackboneBody(2)
        self.offset_predictor = nn.Linear(512, 8)

    def forward(self, x): # (B, 2, 64, 64)
        features = self.body(x) # (B, 512)
        offsets = self.offset_predictor(features) # (B, 8)
        return offsets
```

In [29]:
```
regression_model = RegressionHead().to(device)
regression_optimizer = torch.optim.Adam(regression_model.parameters(),
lr=0.0005)
regression_loss = nn.MSELoss()

print(torchinfo.summary(regression_model, [1, 2, 64, 64], device=device,
                        depth=4,
                        col_names=("input_size", "output_size",
"num_params", "mult_adds"),
                        ))
```

```
==========================================================================
==========================================================================
==
Layer (type:depth-idx)                   Input Shape
Output Shape              Param #                    Mult-Adds
==========================================================================
==========================================================================
==
RegressionHead                           [1, 2, 64, 64]
[1, 8]                    --                         --
├─BackboneBody: 1-1                      [1, 2, 64, 64]
[1, 512]                  --                         --
│    └─Sequential: 2-1                   [1, 2, 64, 64]
[1, 512]                  --                         --
│    │    └─ResNetBlock: 3-1             [1, 2, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    │    └─Conv2d: 4-1             [1, 2, 64, 64]
[1, 64, 64, 64]           192                        786,432
│    │    │    └─Conv2d: 4-2             [1, 2, 64, 64]
[1, 64, 64, 64]           1,152                      4,718,592
│    │    │    └─BatchNorm2d: 4-3        [1, 64, 64, 64]
[1, 64, 64, 64]           128                        128
│    │    │    └─ReLU: 4-4               [1, 64, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    │    └─Conv2d: 4-5             [1, 64, 64, 64]
[1, 64, 64, 64]           36,864                     150,994,944
│    │    │    └─BatchNorm2d: 4-6        [1, 64, 64, 64]
[1, 64, 64, 64]           128                        128
│    │    │    └─ReLU: 4-7               [1, 64, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    └─ResNetBlock: 3-2             [1, 64, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    │    └─Identity: 4-8           [1, 64, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    │    └─Conv2d: 4-9             [1, 64, 64, 64]
[1, 64, 64, 64]           36,864                     150,994,944
│    │    │    └─BatchNorm2d: 4-10       [1, 64, 64, 64]
[1, 64, 64, 64]           128                        128
│    │    │    └─ReLU: 4-11              [1, 64, 64, 64]
[1, 64, 64, 64]           --                         --
│    │    │    └─Conv2d: 4-12            [1, 64, 64, 64]
[1, 64, 64, 64]           36,864                     150,994,944
```

```
│    │    │    └─BatchNorm2d: 4-13       [1, 64, 64, 64]
[1, 64, 64, 64]           128                       128
│    │    │    └─ReLU: 4-14              [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    └─MaxPool2d: 3-3               [1, 64, 64, 64]
[1, 64, 32, 32]           --                        --
│    │    └─ResNetBlock: 3-4             [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Identity: 4-15          [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Conv2d: 4-16            [1, 64, 32, 32]
[1, 64, 32, 32]           36,864                    37,748,736
│    │    │    └─BatchNorm2d: 4-17       [1, 64, 32, 32]
[1, 64, 32, 32]           128                       128
│    │    │    └─ReLU: 4-18              [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Conv2d: 4-19            [1, 64, 32, 32]
[1, 64, 32, 32]           36,864                    37,748,736
│    │    │    └─BatchNorm2d: 4-20       [1, 64, 32, 32]
[1, 64, 32, 32]           128                       128
│    │    │    └─ReLU: 4-21              [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    └─ResNetBlock: 3-5             [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Identity: 4-22          [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Conv2d: 4-23            [1, 64, 32, 32]
[1, 64, 32, 32]           36,864                    37,748,736
│    │    │    └─BatchNorm2d: 4-24       [1, 64, 32, 32]
[1, 64, 32, 32]           128                       128
│    │    │    └─ReLU: 4-25              [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    │    └─Conv2d: 4-26            [1, 64, 32, 32]
[1, 64, 32, 32]           36,864                    37,748,736
│    │    │    └─BatchNorm2d: 4-27       [1, 64, 32, 32]
[1, 64, 32, 32]           128                       128
│    │    │    └─ReLU: 4-28              [1, 64, 32, 32]
[1, 64, 32, 32]           --                        --
│    │    └─MaxPool2d: 3-6               [1, 64, 32, 32]
[1, 64, 16, 16]           --                        --
│    │    └─ResNetBlock: 3-7             [1, 64, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    │    └─Conv2d: 4-29            [1, 64, 16, 16]
[1, 128, 16, 16]          8,320                     2,129,920
│    │    │    └─Conv2d: 4-30            [1, 64, 16, 16]
[1, 128, 16, 16]          73,728                    18,874,368
│    │    │    └─BatchNorm2d: 4-31       [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
│    │    │    └─ReLU: 4-32              [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    │    └─Conv2d: 4-33            [1, 128, 16, 16]
[1, 128, 16, 16]          147,456                   37,748,736
│    │    │    └─BatchNorm2d: 4-34       [1, 128, 16, 16]
```

```
[1, 128, 16, 16]          256                       256
|    |    |    └─ReLU: 4-35             [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
|    |    └─ResNetBlock: 3-8            [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
|    |    |    └─Identity: 4-36         [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
|    |    |    └─Conv2d: 4-37           [1, 128, 16, 16]
[1, 128, 16, 16]          147,456                   37,748,736
|    |    |    └─BatchNorm2d: 4-38      [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
|    |    |    └─ReLU: 4-39             [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
|    |    |    └─Conv2d: 4-40           [1, 128, 16, 16]
[1, 128, 16, 16]          147,456                   37,748,736
|    |    |    └─BatchNorm2d: 4-41      [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
|    |    |    └─ReLU: 4-42             [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
|    |    └─MaxPool2d: 3-9              [1, 128, 16, 16]
[1, 128, 8, 8]            --                        --
|    |    └─ResNetBlock: 3-10           [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Identity: 4-43         [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Conv2d: 4-44           [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
|    |    |    └─BatchNorm2d: 4-45      [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
|    |    |    └─ReLU: 4-46             [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Conv2d: 4-47           [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
|    |    |    └─BatchNorm2d: 4-48      [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
|    |    |    └─ReLU: 4-49             [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    └─ResNetBlock: 3-11           [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Identity: 4-50         [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Conv2d: 4-51           [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
|    |    |    └─BatchNorm2d: 4-52      [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
|    |    |    └─ReLU: 4-53             [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
|    |    |    └─Conv2d: 4-54           [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
|    |    |    └─BatchNorm2d: 4-55      [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
|    |    |    └─ReLU: 4-56             [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
```

```
│    │    └─Flatten: 3-12                    [1, 128, 8, 8]
[1, 8192]                 --                          --
│    │    └─Linear: 3-13                     [1, 8192]
[1, 512]                  4,194,816                   4,194,816
│    │    └─ReLU: 3-14                       [1, 512]
[1, 512]                  --                          --
├─Linear: 1-2                                [1, 512]
[1, 8]                    4,104                       4,104
==========================================================================
==========================================================================
==
Total params: 5,575,624
Trainable params: 5,575,624
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 785.69
==========================================================================
==========================================================================
==
Input size (MB): 0.03
Forward/backward pass size (MB): 25.96
Params size (MB): 22.30
Estimated Total Size (MB): 48.29
==========================================================================
==========================================================================
==
```

Classification head

The classification head takes the 512-dimensional feature vector and instead of predicting continuous values, it treats each of the 8 offsets as a classification problem over 21 discrete bins covering the range [-16, +16]. The output is reshaped to (8, 21) and softmax is applied over the 21 bins for each offset. At inference time the final prediction is computed as a weighted sum (expected value) of the bin centers, converting the discrete distribution back into a continuous offset value. It is trained with cross-entropy loss.

In [30]:

```
class ClassificationHead(nn.Module):
    def __init__(self):
        super().__init__()
        self.body = BackboneBody(2)
        self.logits_layer = nn.Linear(512, 21 * 8) # 168

    def forward(self, x): # (B, 2, 64, 64)
        features = self.body(x) # (B, 512)
        x = self.logits_layer(features) # (B, 168)
        x = x.view(-1, 8, 21) # (B, 8, 21)
        return x
```

In [31]:
```
classification_model = ClassificationHead().to(device)
classification_optimizer =
torch.optim.Adam(classification_model.parameters(), lr=0.0005)
classification_loss = nn.CrossEntropyLoss()

print(torchinfo.summary(classification_model, [1, 2, 64, 64],
device=device,
                        depth=4,
                        col_names=("input_size", "output_size",
"num_params", "mult_adds"),
                        ))
```

```
==========================================================================
==========================================================================
==
Layer (type:depth-idx)                   Input Shape
Output Shape              Param #                   Mult-Adds
==========================================================================
==========================================================================
==
ClassificationHead                       [1, 2, 64, 64]
[1, 8, 21]                --                        --
├─BackboneBody: 1-1                      [1, 2, 64, 64]
[1, 512]                  --                        --
│    └─Sequential: 2-1                   [1, 2, 64, 64]
[1, 512]                  --                        --
│    │    └─ResNetBlock: 3-1             [1, 2, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    │    └─Conv2d: 4-1             [1, 2, 64, 64]
[1, 64, 64, 64]           192                       786,432
│    │    │    └─Conv2d: 4-2             [1, 2, 64, 64]
[1, 64, 64, 64]           1,152                     4,718,592
│    │    │    └─BatchNorm2d: 4-3        [1, 64, 64, 64]
[1, 64, 64, 64]           128                       128
│    │    │    └─ReLU: 4-4               [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    │    └─Conv2d: 4-5             [1, 64, 64, 64]
[1, 64, 64, 64]           36,864                    150,994,944
│    │    │    └─BatchNorm2d: 4-6        [1, 64, 64, 64]
[1, 64, 64, 64]           128                       128
│    │    │    └─ReLU: 4-7               [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    └─ResNetBlock: 3-2             [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    │    └─Identity: 4-8           [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    │    └─Conv2d: 4-9             [1, 64, 64, 64]
[1, 64, 64, 64]           36,864                    150,994,944
│    │    │    └─BatchNorm2d: 4-10       [1, 64, 64, 64]
[1, 64, 64, 64]           128                       128
│    │    │    └─ReLU: 4-11              [1, 64, 64, 64]
[1, 64, 64, 64]           --                        --
│    │    │    └─Conv2d: 4-12            [1, 64, 64, 64]
```

```
[1, 64, 64, 64]          36,864                    150,994,944
|    |    |    └─BatchNorm2d: 4-13       [1, 64, 64, 64]
[1, 64, 64, 64]          128                       128
|    |    |    └─ReLU: 4-14              [1, 64, 64, 64]
[1, 64, 64, 64]          --                        --
|    |    └─MaxPool2d: 3-3               [1, 64, 64, 64]
[1, 64, 32, 32]          --                        --
|    |    └─ResNetBlock: 3-4             [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Identity: 4-15          [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Conv2d: 4-16            [1, 64, 32, 32]
[1, 64, 32, 32]          36,864                    37,748,736
|    |    |    └─BatchNorm2d: 4-17       [1, 64, 32, 32]
[1, 64, 32, 32]          128                       128
|    |    |    └─ReLU: 4-18              [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Conv2d: 4-19            [1, 64, 32, 32]
[1, 64, 32, 32]          36,864                    37,748,736
|    |    |    └─BatchNorm2d: 4-20       [1, 64, 32, 32]
[1, 64, 32, 32]          128                       128
|    |    |    └─ReLU: 4-21              [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    └─ResNetBlock: 3-5             [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Identity: 4-22          [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Conv2d: 4-23            [1, 64, 32, 32]
[1, 64, 32, 32]          36,864                    37,748,736
|    |    |    └─BatchNorm2d: 4-24       [1, 64, 32, 32]
[1, 64, 32, 32]          128                       128
|    |    |    └─ReLU: 4-25              [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    |    └─Conv2d: 4-26            [1, 64, 32, 32]
[1, 64, 32, 32]          36,864                    37,748,736
|    |    |    └─BatchNorm2d: 4-27       [1, 64, 32, 32]
[1, 64, 32, 32]          128                       128
|    |    |    └─ReLU: 4-28              [1, 64, 32, 32]
[1, 64, 32, 32]          --                        --
|    |    └─MaxPool2d: 3-6               [1, 64, 32, 32]
[1, 64, 16, 16]          --                        --
|    |    └─ResNetBlock: 3-7             [1, 64, 16, 16]
[1, 128, 16, 16]         --                        --
|    |    |    └─Conv2d: 4-29            [1, 64, 16, 16]
[1, 128, 16, 16]         8,320                     2,129,920
|    |    |    └─Conv2d: 4-30            [1, 64, 16, 16]
[1, 128, 16, 16]         73,728                    18,874,368
|    |    |    └─BatchNorm2d: 4-31       [1, 128, 16, 16]
[1, 128, 16, 16]         256                       256
|    |    |    └─ReLU: 4-32              [1, 128, 16, 16]
[1, 128, 16, 16]         --                        --
|    |    |    └─Conv2d: 4-33            [1, 128, 16, 16]
[1, 128, 16, 16]         147,456                   37,748,736
```

```
│    │    │    └─BatchNorm2d: 4-34       [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
│    │    │    └─ReLU: 4-35              [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    └─ResNetBlock: 3-8             [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    │    └─Identity: 4-36          [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    │    └─Conv2d: 4-37            [1, 128, 16, 16]
[1, 128, 16, 16]          147,456                   37,748,736
│    │    │    └─BatchNorm2d: 4-38       [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
│    │    │    └─ReLU: 4-39              [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    │    └─Conv2d: 4-40            [1, 128, 16, 16]
[1, 128, 16, 16]          147,456                   37,748,736
│    │    │    └─BatchNorm2d: 4-41       [1, 128, 16, 16]
[1, 128, 16, 16]          256                       256
│    │    │    └─ReLU: 4-42              [1, 128, 16, 16]
[1, 128, 16, 16]          --                        --
│    │    └─MaxPool2d: 3-9               [1, 128, 16, 16]
[1, 128, 8, 8]            --                        --
│    │    └─ResNetBlock: 3-10            [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Identity: 4-43          [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Conv2d: 4-44            [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
│    │    │    └─BatchNorm2d: 4-45       [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
│    │    │    └─ReLU: 4-46              [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Conv2d: 4-47            [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
│    │    │    └─BatchNorm2d: 4-48       [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
│    │    │    └─ReLU: 4-49              [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    └─ResNetBlock: 3-11            [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Identity: 4-50          [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Conv2d: 4-51            [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
│    │    │    └─BatchNorm2d: 4-52       [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
│    │    │    └─ReLU: 4-53              [1, 128, 8, 8]
[1, 128, 8, 8]            --                        --
│    │    │    └─Conv2d: 4-54            [1, 128, 8, 8]
[1, 128, 8, 8]            147,456                   9,437,184
│    │    │    └─BatchNorm2d: 4-55       [1, 128, 8, 8]
[1, 128, 8, 8]            256                       256
│    │    │    └─ReLU: 4-56              [1, 128, 8, 8]
```

```
[1, 128, 8, 8]            --                        --
│    │    └─Flatten: 3-12                [1, 128, 8, 8]
[1, 8192]                 --                        --
│    │    └─Linear: 3-13                 [1, 8192]
[1, 512]                  4,194,816                 4,194,816
│    │    └─ReLU: 3-14                   [1, 512]
[1, 512]                  --                        --
├─Linear: 1-2                           [1, 512]
[1, 168]                  86,184                    86,184
=================================================================
=================================================================
==
Total params: 5,657,704
Trainable params: 5,657,704
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 785.77
=================================================================
=================================================================
==
Input size (MB): 0.03
Forward/backward pass size (MB): 25.96
Params size (MB): 22.63
Estimated Total Size (MB): 48.62
=================================================================
=================================================================
==
```

Train and result visualization
tensorboard --logdir=runs
in the terminal

In [32]:
```
def train(model, loader, optimizer, loss_fn, device,
is_classification=False):
    model.train()
    total_loss = 0
    steps = 0

    for sample, offsets, base_corners, moved_corners, H_mat, warped_image
in loader:
        sample  = sample.to(device)
        offsets = offsets.to(device)

        optimizer.zero_grad()
        prediction = model(sample)

        if is_classification: # offsets -> (B, 4, 2)
            target = offsets.view(-1, 8) # (B, 8)
            target = ((target + 16) / 32 * 20).round().long() # class
indexes
            target = torch.clamp(target, 0, 20)
            prediction = prediction.permute(0, 2, 1) # (B, 8, 21) -> (B,
21, 8)
            loss = loss_fn(prediction, target)
        else:
            target = offsets.view(-1, 8) # (B, 8)
            prediction = prediction.view(-1, 8) # # (B, 8)
            loss = loss_fn(prediction, target)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        steps += 1

    return total_loss / steps
```

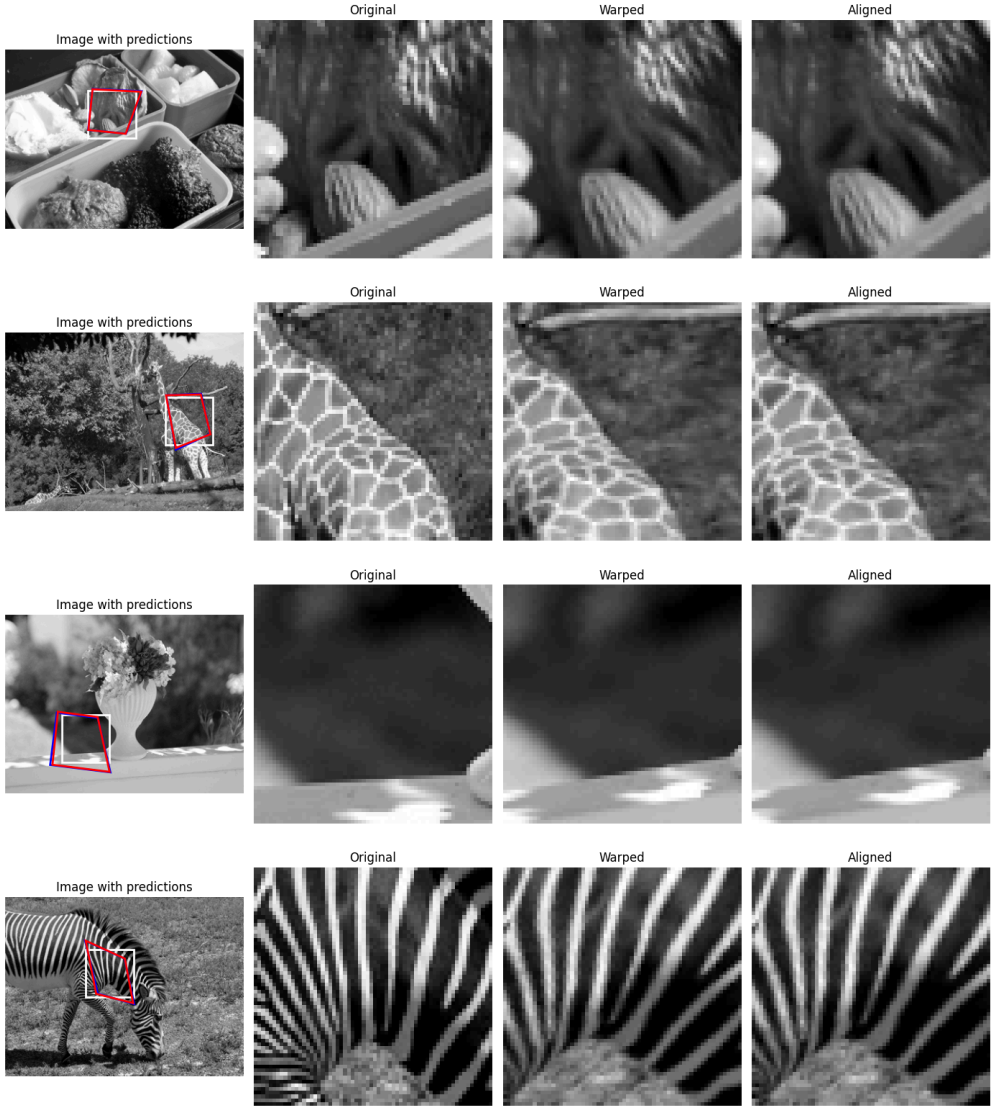
In [33]:
```
def plot_loss(loss_list, window):
    loss = np.array(loss_list, dtype=float)

    if len(loss) >= window:
        shape = (loss.size - window + 1, window)
        strides = (loss.strides[0], loss.strides[0])
        windows = np.lib.stride_tricks.as_strided(loss, shape=shape,
strides=strides)
        smooth = windows.mean(axis=1)
    else:
        smooth = loss

    plt.figure(figsize=(8,4))
    plt.plot(loss, label="Raw loss")
    plt.plot(range(window-1, window-1 + len(smooth)), smooth,
label=f"Smoothed (window={window})")
    plt.xlabel("Step")
    plt.ylabel("Loss")
    plt.title("Training Loss")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Regression

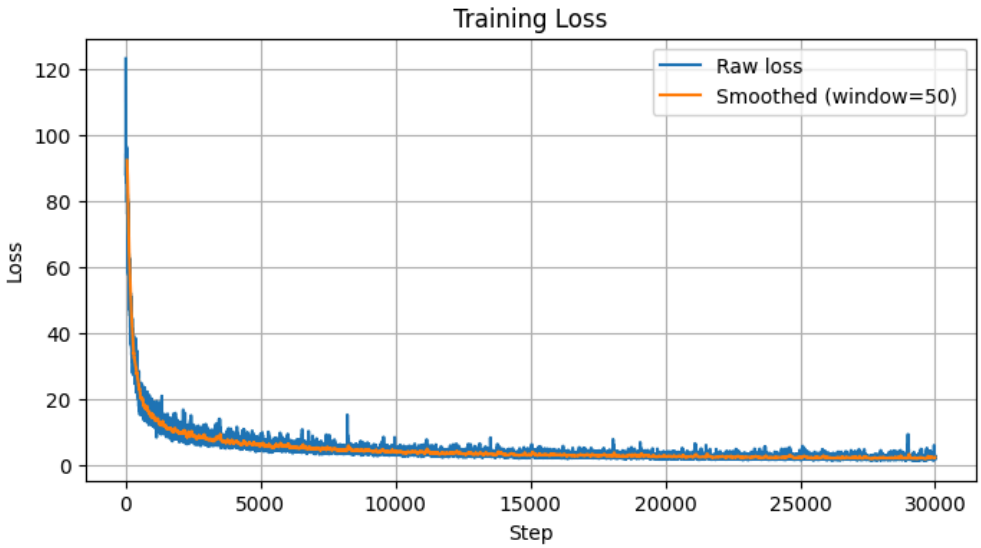
In [ ]:
```
regression_losses = []
start_time = time.time()

epochs = 30000

for epoch in range(epochs):
    epoch_start = time.time()
    train_loss= train(
        regression_model, dataloader, regression_optimizer,
        regression_loss, device,
        is_classification=False
    )
    regression_losses.append(train_loss)

    epoch_ms = (time.time() - epoch_start) * 1000

    print(f'[Epoch {epoch:>3}/{epochs}] | Loss: {train_loss:.4f} | Epoch
time: {epoch_ms:.1f}ms ')

torch.save({
            'epoch': epochs,
            'model_state_dict': regression_model.state_dict(),
            'optimizer_state_dict': regression_optimizer.state_dict(),
            'losses': regression_losses,
        }, 'regression_final.pth')
```

In [35]:
```
def visualize_regression_results( n=4):
    regression_model.eval()

    fig, axes = plt.subplots(n, 4, figsize=(14, n * 4))

    for i in range(n):
        image = dataset.images[i]
        data = dataset.generate_image(image)

        sample        = data['sample'].unsqueeze(0).to(device)  # (1, 2,
64, 64)
        base_corners  = data['base_corners'].numpy()             # (4, 2)
        moved_corners = data['moved_corners'].numpy()            # (4, 2)
        offsets_gt    = data['offsets'].numpy()                  # (4, 2)
        patch        = data['sample'][0].numpy()  # org
        warped_patch = data['sample'][1].numpy()

        with torch.no_grad():
            prediction = regression_model(sample).cpu().numpy().reshape(4,
2)  # (4, 2)

        predicted_corners = base_corners + prediction

        H_pred = cv2.getPerspectiveTransform(
            base_corners.astype(np.float32),
            predicted_corners.astype(np.float32)
        ) # homography on model res
        try:
            H_pred_inv = np.linalg.inv(H_pred)
            image_H, image_W = image.shape
            aligned_image = cv2.warpPerspective(image, H_pred_inv,
(image_W, image_H))
            x = int(base_corners[0, 0])
            y = int(base_corners[0, 1])
            aligned_patch = aligned_image[y:y + dataset.patch_size, x:x +
dataset.patch_size]
        except:
            aligned_patch = np.zeros_like(patch)

        x = int(base_corners[0, 0])
        y = int(base_corners[0, 1])

        axes[i, 0].imshow(image, cmap='gray')

        white_rect = plt.Rectangle((x, y), dataset.patch_size,
dataset.patch_size,linewidth=2, edgecolor='white', facecolor='none') # org
patch
        axes[i, 0].add_patch(white_rect)

        blue_poly = plt.Polygon(moved_corners, linewidth=2,
edgecolor='blue', facecolor='none', closed=True)
        axes[i, 0].add_patch(blue_poly) # moved corners
```

```
        red_poly = plt.Polygon(predicted_corners, linewidth=2,
edgecolor='red', facecolor='none', closed=True)
        axes[i, 0].add_patch(red_poly) # model res (pred)

        axes[i, 0].set_title('Image with predictions')
        axes[i, 0].axis('off')

        axes[i, 1].imshow(patch, cmap='gray')
        axes[i, 1].set_title('Original')
        axes[i, 1].axis('off')

        axes[i, 2].imshow(warped_patch, cmap='gray')
        axes[i, 2].set_title('Warped')
        axes[i, 2].axis('off')

        axes[i, 3].imshow(aligned_patch, cmap='gray')
        axes[i, 3].set_title('Aligned')
        axes[i, 3].axis('off')

    plt.tight_layout()
    plt.show()

visualize_regression_results(n=4)
```



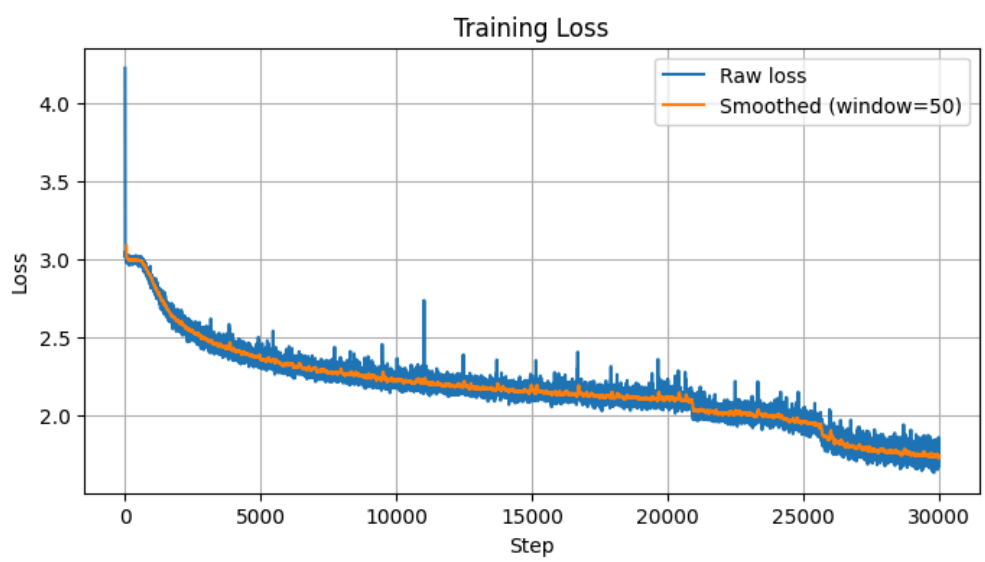
In [36]:
```
plot_loss(regression_losses, window=50)
```



Classification

In [ ]:
```
classification_losses = []
start_time = time.time()

for epoch in range(epochs):
    epoch_start = time.time()
    train_loss = train(
        classification_model, dataloader, classification_optimizer,
        classification_loss, device,
        is_classification=True
    )
    classification_losses.append(train_loss)

    epoch_ms = (time.time() - epoch_start) * 1000

    print(f'[Epoch {epoch:>3}/{epochs}] | Loss: {train_loss:.4f} | Epoch
time: {epoch_ms:.1f}ms ')

torch.save({
    'epoch': epochs,
    'model_state_dict': classification_model.state_dict(),
    'optimizer_state_dict': classification_optimizer.state_dict(),
    'losses': classification_losses,
}, 'classification_final.pth')
```

In [38]:
```
plot_loss(classification_losses, window=50)
```



In [39]:
```
def visualize_classification_predictions(n=4):
    classification_model.eval()

    bin_centers = torch.linspace(-16, 16, 21).to(device) # 21 bins in [-16,
16]

    fig, axes = plt.subplots(n, 4, figsize=(14, n * 4))

    for i in range(n):
        image = dataset.images[i]
        data = dataset.generate_image(image)

        sample        = data['sample'].unsqueeze(0).to(device)  # (1, 2,
64, 64)
        base_corners  = data['base_corners'].numpy()             # (4, 2)
        moved_corners = data['moved_corners'].numpy()            # (4, 2)
        offsets_gt    = data['offsets'].numpy()                  # (4, 2)
        patch        = data['sample'][0].numpy()
        warped_patch = data['sample'][1].numpy()

        with torch.no_grad():
            logits = classification_model(sample)             # (1, 8, 21)
            probabilities = torch.softmax(logits, dim=2)              # (1,
8, 21) probability on 21 bins
            predicted_offsets = (probabilities * bin_centers).sum(dim=2)
# (1, 8)
            prediction = predicted_offsets.cpu().numpy().reshape(4, 2)  #
(4, 2)

        predicted_corners = base_corners + prediction

        H_pred = cv2.getPerspectiveTransform(
            base_corners.astype(np.float32),
            predicted_corners.astype(np.float32)
        ) # homography on model res
        try:
            H_pred_inv = np.linalg.inv(H_pred)
            image_H, image_W = image.shape
            aligned_image = cv2.warpPerspective(image, H_pred_inv,
(image_W, image_H))
            x = int(base_corners[0, 0])
            y = int(base_corners[0, 1])
            aligned_patch = aligned_image[y:y + dataset.patch_size, x:x +
dataset.patch_size]
        except:
            aligned_patch = np.zeros_like(patch)

        x = int(base_corners[0, 0])
        y = int(base_corners[0, 1])

        axes[i, 0].imshow(image, cmap='gray')

        white_rect = plt.Rectangle((x, y), dataset.patch_size,
```

```
dataset.patch_size,
                                    linewidth=2, edgecolor='white',
facecolor='none')
        axes[i, 0].add_patch(white_rect)

        blue_poly = plt.Polygon(moved_corners, linewidth=2,
edgecolor='blue', facecolor='none', closed=True)
        axes[i, 0].add_patch(blue_poly)

        red_poly = plt.Polygon(predicted_corners, linewidth=2,
edgecolor='red', facecolor='none', closed=True)
        axes[i, 0].add_patch(red_poly)

        axes[i, 0].set_title('Image with predictions (classification)')
        axes[i, 0].axis('off')

        axes[i, 1].imshow(patch, cmap='gray')
        axes[i, 1].set_title('Original')
        axes[i, 1].axis('off')

        axes[i, 2].imshow(warped_patch, cmap='gray')
        axes[i, 2].set_title('Warped')
        axes[i, 2].axis('off')

        axes[i, 3].imshow(aligned_patch, cmap='gray')
        axes[i, 3].set_title('Aligned')
        axes[i, 3].axis('off')

    plt.tight_layout()
    plt.show()


visualize_classification_predictions(n=4)
```

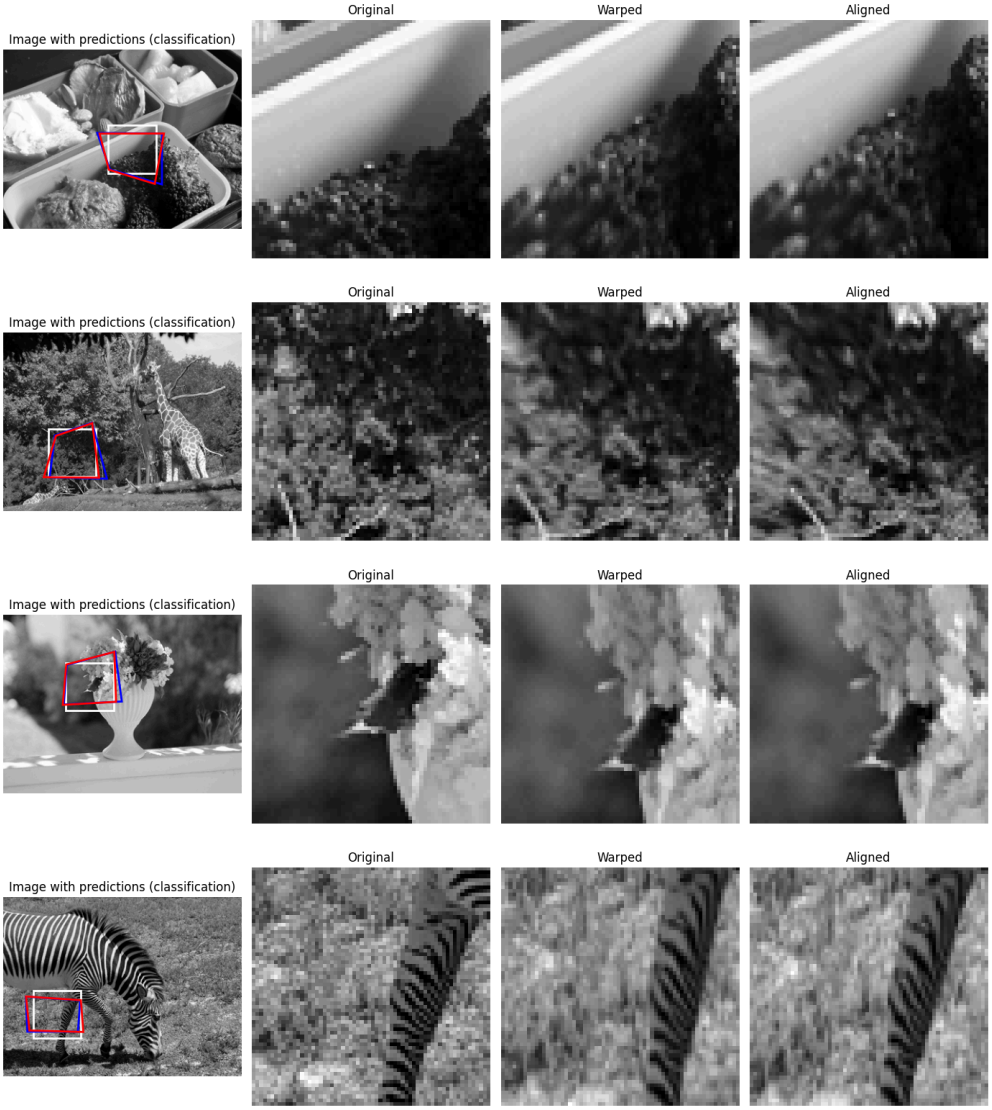
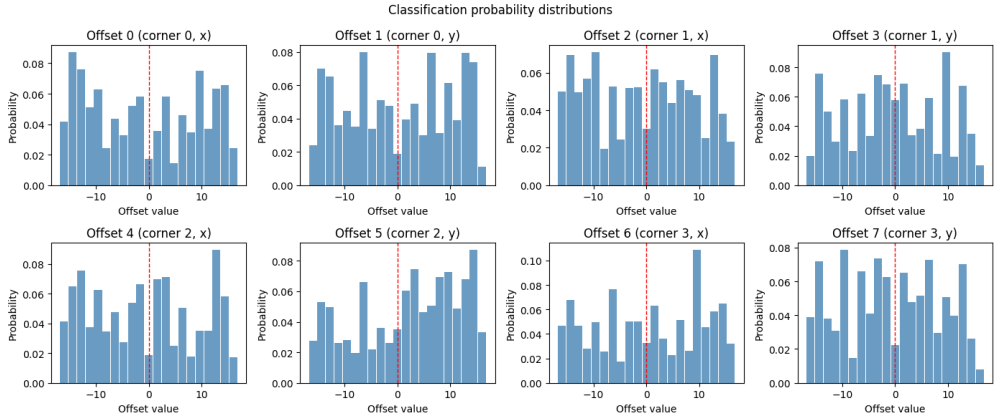


Image with predictions (classification)
Original
Warped
Aligned
Image with predictions (classification)
Original
Warped
Aligned
Image with predictions (classification)
Original
Warped
Aligned
Image with predictions (classification)
Original
Warped
Aligned

In [40]:
```
def visualize_classification_results(n_samples=100):
    classification_model.eval()

    fig, axes = plt.subplots(2, 4, figsize=(14, 6))
    axes = axes.flatten()

    all_probabilities = [[] for _ in range(8)]
    bin_centers = np.linspace(-16, 16, 21) # real vals for 21 bins

    with torch.no_grad():
        for i in range(n_samples):
            image = dataset.images[i % len(dataset.images)]
            data = dataset.generate_image(image)
            sample = data['sample'].unsqueeze(0).to(device)

            prediction = classification_model(sample) # (1, 8, 21)
            probs = torch.softmax(prediction, dim=2)  # (1, 8, 21)
            probs = probs.squeeze(0).cpu().numpy()   # (8, 21)

            for j in range(8):
                all_probabilities[j].append(probs[j]) # trough 100ex, for
each of 8 coord saves the selected bin from model

    for j in range(8):
        avg_prob = np.mean(all_probabilities[j], axis=0)  # (21,)

        axes[j].bar(bin_centers, avg_prob, width=1.5, color='steelblue',
alpha=0.8)
        axes[j].axvline(0, color='red', linewidth=1, linestyle='--')
        axes[j].set_title(f'Offset {j} (corner {j//2}, {"x" if j%2==0 else
"y"})')
        axes[j].set_xlabel('Offset value')
        axes[j].set_ylabel('Probability')

    plt.suptitle('Classification probability distributions')
    plt.tight_layout()
    plt.show()


visualize_classification_results(n_samples=100)
```



Evaluation on different images
In training the DataLoader randomly picks images, so the same image can be picked multiple times or skipped entirely in one epoch.

In evaluation you loop directly and generate exactly 10 samples per image, so every image is used exactly 10 times, giving you a controlled and reproducible test set.
Essentially:

Training — random, repetitive, driven by DataLoader
Evaluation — systematic, every image used exactly N times

In [50]:
```
test_dataset = HomographyPreloadDataset(root="'./dataset_test")
test_dataset_classical = HomographyPreloadDataset(
    root="'./dataset_test",
    resize=(640, 480),
    patch_size=256,
    max_shift=64,
)
def evaluate(model, dataset, device, n_per_image=10,
is_classification=False):
    model.eval()
    errors = []
    bin_centers = torch.linspace(-16, 16, 21).to(device)

    with torch.no_grad():
        for image in dataset.images:
            for _ in range(n_per_image):
                data = dataset.generate_image(image)

                sample     = data['sample'].unsqueeze(0).to(device)
                offsets_gt = data['offsets'].numpy().flatten()  # (8,)

                prediction = model(sample)

                if is_classification:
                    probabilities = torch.softmax(prediction, dim=2)
                    predicted_offsets = (probabilities *
bin_centers).sum(dim=2)
                    predicted_offsets =
predicted_offsets.cpu().numpy().flatten()
                else:
                    predicted_offsets = prediction.cpu().numpy().flatten()

                rmse = np.sqrt(np.mean((predicted_offsets - offsets_gt) **
2))
                errors.append(rmse)

    errors = np.array(errors)
    print(f"Samples: {len(errors)}")
    print(f"Mean:    {errors.mean():.4f}")
    print(f"Median:  {np.median(errors):.4f}")
    print(f"Std:     {errors.std():.4f}")
    return errors

def plot_evaluation(regression_errors, classification_errors):
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # boxplot
    axes[0].boxplot([regression_errors, classification_errors], labels=
['Regression', 'Classification'])
    axes[0].set_ylabel('RMSE')
    axes[0].set_title('Error distribution (boxplot)')
    axes[0].grid(True)
```

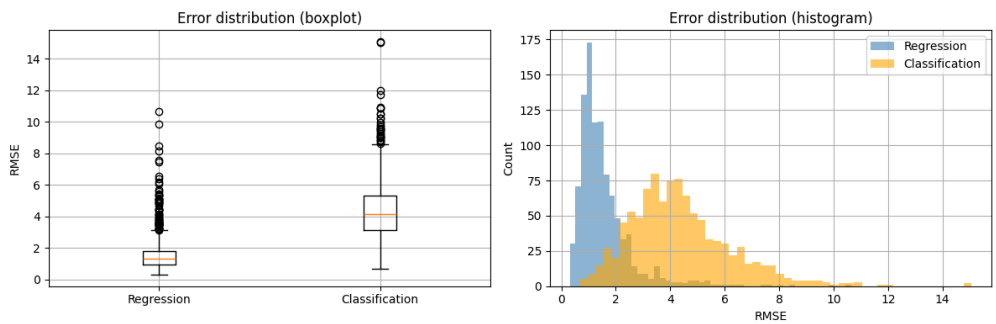
```
    # histogram
    axes[1].hist(regression_errors, bins=50, alpha=0.6, label='Regression',
color='steelblue')
    axes[1].hist(classification_errors, bins=50, alpha=0.6,
label='Classification', color='orange')
    axes[1].set_xlabel('RMSE')
    axes[1].set_ylabel('Count')
    axes[1].set_title('Error distribution (histogram)')
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
```

```
Preloading 100 samples into RAM...
  10/100 loaded...
  20/100 loaded...
  30/100 loaded...
  40/100 loaded...
  50/100 loaded...
  60/100 loaded...
  70/100 loaded...
  80/100 loaded...
  90/100 loaded...
  100/100 loaded...
Done
Preloading 100 samples into RAM...
  10/100 loaded...
  20/100 loaded...
  30/100 loaded...
  40/100 loaded...
  50/100 loaded...
  60/100 loaded...
  70/100 loaded...
  80/100 loaded...
  90/100 loaded...
  100/100 loaded...
Done
```

In [51]:
```
regression_errors = evaluate(regression_model, test_dataset, device,
is_classification=False)
classification_errors = evaluate(classification_model, test_dataset,
device, is_classification=True)

plot_evaluation(regression_errors, classification_errors)
```

```
Samples: 1000
Mean:    1.5703
Median:  1.2920
Std:     1.0715
Samples: 1000
Mean:    4.3887
Median:  4.1169
Std:     1.8875
```

```
/tmp/ipykernel_1258446/785857353.py:44:
MatplotlibDeprecationWarning: The 'labels' parameter of boxplot()
has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  axes[0].boxplot([regression_errors, classification_errors],
labels=['Regression', 'Classification'])
```



Comparing to classical methods

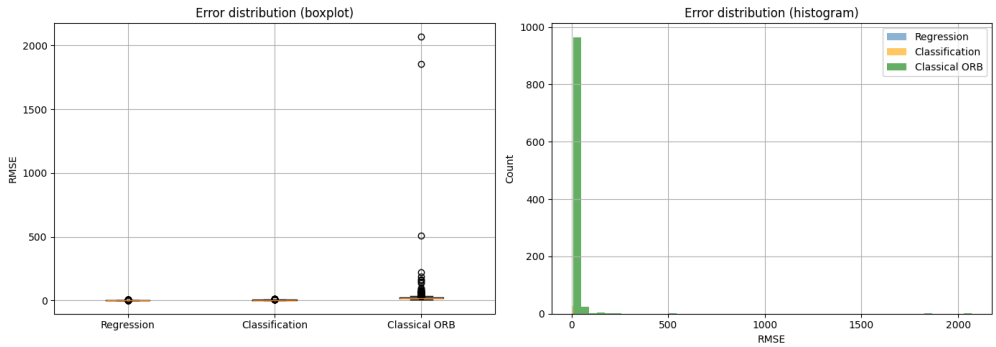
In [54]:

```
def evaluate_classical(dataset, n_per_image=10):
    errors = []
    orb = cv2.ORB_create(nfeatures=1000)
    matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    for image in dataset.images:
        for _ in range(n_per_image):
            data = dataset.generate_image(image)

            offsets_gt = data['offsets'].numpy().flatten()  # (8,)

            patch_uint8  = (data['sample'][0].numpy() *
255).astype(np.uint8)
            warped_uint8 = (data['sample'][1].numpy() *
255).astype(np.uint8)

            keypoints1, descriptor1 = orb.detectAndCompute(patch_uint8,
None)
            keypoints2, descriptor2 = orb.detectAndCompute(warped_uint8,
None)

            H_classical = None
            if len(descriptor1) >= 4 and len(descriptor2) >= 4:
                matches = matcher.match(descriptor1, descriptor2)
                if len(matches) >= 4:
                    pts1 = np.float32([keypoints1[m.queryIdx].pt for m in
matches])
                    pts2 = np.float32([keypoints2[m.trainIdx].pt for m in
matches])
                    H_classical, _ = cv2.findHomography(pts1, pts2,
cv2.RANSAC)


            patch_size_orb = dataset.patch_size  # 256
            base_corners_orb = np.array(
                [[0, 0], [patch_size_orb - 1, 0], [patch_size_orb - 1,
patch_size_orb - 1], [0, patch_size_orb - 1]],
                dtype=np.float32,
            )

            corners_homography = cv2.perspectiveTransform(
                base_corners_orb.reshape(1, -1, 2), H_classical
            ).reshape(4, 2)

            predicted_offsets_orb = (corners_homography -
base_corners_orb).flatten()  # (8,)

            rmse_256 = np.sqrt(np.mean((predicted_offsets_orb - offsets_gt)
** 2))

            errors.append(rmse_256 / 4.0) # 256/64 = 4

    errors = np.array(errors)
```

```
    print(f"Classical ORB — Samples: {len(errors)}")
    print(f"Mean:   {errors.mean():.4f}")
    print(f"Median: {np.median(errors):.4f}")
    print(f"Std:    {errors.std():.4f}")
    return errors

def plot_all_methods(reg_errors, cls_errors, classical_errors):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # boxplot
    axes[0].boxplot(
        [reg_errors, cls_errors, classical_errors],
        labels=['Regression', 'Classification', 'Classical ORB']
    )
    axes[0].set_ylabel('RMSE')
    axes[0].set_title('Error distribution (boxplot)')
    axes[0].grid(True)

    # histogram
    axes[1].hist(reg_errors,       bins=50, alpha=0.6, label='Regression',
color='steelblue')
    axes[1].hist(cls_errors,       bins=50, alpha=0.6,
label='Classification',    color='orange')
    axes[1].hist(classical_errors, bins=50, alpha=0.6, label='Classical
ORB',     color='green')
    axes[1].set_xlabel('RMSE')
    axes[1].set_ylabel('Count')
    axes[1].set_title('Error distribution (histogram)')
    axes[1].legend()
    axes[1].grid(True)

    plt.tight_layout()
    plt.show()
```

In [55]:
```
classical_errors = evaluate_classical(test_dataset_classical,
n_per_image=10)
plot_all_methods(regression_errors, classification_errors,
classical_errors)
```

```
Classical ORB — Samples: 1000
Mean:   27.6083
Median: 20.0829
Std:    89.4928
```



Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.